

Unit 2: Python Control Flow, Functions & Modules • Topic 3

Break, Continue and Pass

Controlling loop execution precisely using the break, continue and pass statements.

A loop normally runs every iteration, in full, until its condition becomes False. Sometimes we need finer control — to stop a loop early, skip just one iteration, or write a placeholder that does nothing (yet). Python provides three keywords for this: break, continue, and pass.

3.1 The break Statement

break immediately **terminates the innermost loop** it is placed in — execution jumps straight to the first statement after the loop, skipping any remaining iterations entirely.

```
for i in range(1, 10):
    if i == 5:
        break          # loop stops completely when i becomes 5
    print(i)
# Output: 1 2 3 4
print("Loop ended")
```

Typical use: searching for a value and stopping as soon as it is found.

```
numbers = [4, 9, 15, 22, 7]
target = 15
for n in numbers:
    if n == target:
        print("Found it!")
        break
else:
    print("Not found")
```

3.2 The continue Statement

continue **skips the rest of the current iteration only** and moves straight to the next iteration of the loop — the loop itself keeps running, unlike break.

```
for i in range(1, 6):
    if i == 3:
        continue      # skip printing only when i is 3
    print(i)
# Output: 1 2 4 5    (3 is skipped, but the loop keeps going)
```

Typical use: skipping unwanted values while still processing the rest of a sequence.

```
for n in range(1, 11):
    if n % 2 != 0:
        continue      # skip odd numbers
    print(n, end=" ")
# Output: 2 4 6 8 10
```

3.3 The pass Statement

`pass` is a **null operation** — it does absolutely nothing when executed. It is used as a syntactic placeholder wherever Python's syntax requires a statement (e.g. after a colon), but you have not yet decided what code should go there, or genuinely want an empty block.

```
for i in range(5):
    if i == 3:
        pass                # placeholder - does nothing, but keeps the code syntactically valid
    print(i)
# Output: 0 1 2 3 4 (pass has no effect on the flow at all)

def calculate_tax(income):
    pass    # TODO: implement this function later; pass avoids an IndentationError for now
```

WHY IT EXISTS: Without `pass`, writing 'if i == 3:' followed by nothing indented underneath would raise 'IndentationError: expected an indented block' — `pass` is the standard way to leave a block intentionally empty.

3.4 break vs continue vs pass — Side-by-Side Comparison

Statement	Effect on the loop	Remaining iterations?
<code>break</code>	Exits/terminates the loop immediately	All remaining iterations are skipped
<code>continue</code>	Skips only the current iteration's remaining code	Loop continues to the next iteration
<code>pass</code>	Does nothing at all (a no-op placeholder)	No effect on the loop whatsoever

Visual Summary with the Same Loop

```
# for i in range(1, 6): with different keywords when i == 3

break    ->  Output: 1 2                (stops completely at i=3)
continue ->  Output: 1 2 4 5            (skips only i=3, loop continues)
pass     ->  Output: 1 2 3 4 5          (i=3 still printed normally, no effect)
```

Practice Questions

A. Multiple Choice Questions

- Which statement immediately terminates the loop it is inside?
(A) continue (B) pass (C) break (D) return
- Which statement skips only the current iteration and moves to the next one?
(A) break (B) continue (C) pass (D) exit
- What does the pass statement do?
(A) Skips the current iteration (B) Terminates the loop (C) Does nothing; acts as a placeholder (D) Raises an error
- for i in range(1,5): \n if i==2: break \n print(i) -- what is printed?
(A) 1 (B) 1 2 (C) 1 2 3 4 (D) 2 3 4
- for i in range(1,5): \n if i==2: continue \n print(i) -- what is printed?
(A) 1 3 4 (B) 1 2 3 4 (C) 1 (D) 2

B. True / False

- break stops only the current iteration but allows the loop to continue.
- continue causes the loop to skip the remaining statements in the current iteration only.
- pass can be used as a placeholder inside an empty function body.
- break and continue can only be used inside loops (for/while), not inside if statements alone.

C. Short Answer Questions

- Differentiate between break and continue with one code example each.
- Why is the pass statement needed in Python? Give an example where omitting it causes an error.
- Trace the output of: for i in range(1,6): \n if i%2==0: continue \n print(i)
- In what situation would you prefer break over continue while searching a list of numbers?

D. Long Answer / Descriptive Questions

- Write a Python program that reads numbers from a list and uses break to stop as soon as a negative number is encountered, printing all positive numbers read so far.
- Write a Python program that prints numbers from 1 to 20, skipping multiples of 3 using continue, and explain step by step how the flow of control differs from using break in the same place.

E. Programming Problems

- Write a Python program that takes numbers from the user one by one (using a while loop) until the user enters -1 (use break to stop), and prints their sum.
- Write a Python program that prints all numbers from 1 to 50 that are NOT divisible by 3 or 5, using continue to skip the divisible ones.
- Write a Python program with a loop and an empty if-block for a condition you have not yet implemented, using pass so the program still runs without errors.

Answer Key (A & B): MCQ: 1-C, 2-B, 3-C, 4-A, 5-A | T/F: 6-False, 7-True, 8-True, 9-True